Module 08: Planning — Deep Temporal Models and Long-Horizon Inference

Learning Objectives

1. Extend EFE computation to multi-step policies with temporal depth T.
2. Implement marginal message passing (MMP) using `run_mmp()` for deep temporal inference.
3. Build and simulate a gridworld environment with long-horizon planning.

Introduction

In Modules 01–07, the agent operated at single-step or short-horizon timescales. Real-world problems require **planning** — evaluating sequences of actions that unfold over many future steps. Deep temporal models maintain beliefs about states at multiple time points and evaluate policies by accumulating EFE across an entire action sequence.

Key Concepts

1. Multi-Step Policies

A policy $\pi = [a_0, a_1, \ldots, a_{T-1}]$ specifies actions for T future timesteps. The agent evaluates each policy by unrolling the predicted state trajectory:

$$q(s_{\tau+1} \mid \pi) = \mathbf{B}{*a*\tau} \cdot q(s_\tau \mid \pi) \quad \text{for } \tau = 0, \ldots, T-1$$

EFE is accumulated over all steps:

$$G(\pi) = \sum_{\tau=0}^{T-1} G_\tau(\pi)$$

```
# Define multi-step policies for a 3-action system
policies = [
    [0, 0, 0],  # stay, stay, stay
    [1, 0, 0],  # left, stay, stay
    [1, 2, 0],  # left, right, stay
    [2, 1, 0],  # right, left, stay
]

agent = ActiveInferenceAgent(model, gamma=4.0, policies=policies)
```

2. Marginal Message Passing (MMP)

For deep temporal models, `run_mmp()` performs inference over beliefs at multiple time points simultaneously. Instead of just inferring $q(s_t)$, MMP infers $q(s_\tau)$ for $\tau \in {0, 1, \ldots, T}$:

```
from active_inference.math import run_mmp

result = run_mmp(
    prior=model.D,
    observations=[0, 1, 0],     # sequence of past observations
    A=model.A,
    B=model.B,
    policy=[1, 0, 1],           # action sequence
    num_iterations=16,
)

print(result["beliefs"])         # list of belief vectors, one per timest
print(result["converged"])       # convergence status
print(result["delta_history"])   # convergence trace
```

MMP passes messages both forward (prior × transition) and backward (likelihood) to refine beliefs at each time point.

3. Temporal Depth T

The temporal depth T controls how far ahead the agent plans:

T	Behavior
T = 1	Reactive — considers only the next step (Modules 01–05)
T = 2–3	Short-horizon — can sequence actions (e.g., go-to-cue, then go-to-reward)
T = 5+	Deep planning — can navigate gridworlds, handle delayed rewards

Tradeoff: Larger T gives better plans but exponentially increases the number of possible policies (N_a^T for N_a actions).

4. Gridworld Implementation

Gridworlds are the natural testbed for planning. A gridworld has:

- $N \times M$ grid of states (flattened to $N \cdot M$ states)
- 4 or 5 actions (up, down, left, right, stay)
- Walls/obstacles encoded in the B-matrix (self-transitions for blocked moves)
- A goal state with preferred observation via C

```
from active_inference.visualization import plot_gridworld

# Visualize a 4x4 gridworld
grid = np.zeros((4, 4))
obstacles = [(1, 1), (1, 2)]
path = [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3)]
goal = (3, 3)

plot_gridworld(grid, obstacles=obstacles, path=path, goal=goal,
               save_path="output/lab08_gridworld.png")
```

5. T-Maze with Delayed Reward

The T-maze with temporal depth demonstrates the power of planning:

- **T = 1**: Agent cannot sequence cue-then-reward. It picks randomly.
- **T = 2**: Agent can plan: go-to-cue first, then go-to-reward-arm. The cue visit reduces ambiguity at step 1, enabling a risk-minimizing choice at step 2.

6. Policy Evaluation for Deep Models

For deep models, each policy's EFE is the sum over timesteps:

```
from active_inference.math import compute_efe

total_G = 0
q_current = agent.q_s.copy()
for t, action in enumerate(policy):
    G_t = compute_efe(q_current, model.A, model.B, model.C, action)
    total_G += G_t
    q_current = model.B[:, :, action] @ q_current  # predict next-state
```

7. Simulation Dashboard

The `plot_simulation_dashboard()` function provides a 5-panel overview of a complete simulation:

```
 from active_inference.visualization import plot_simulation_dashboard

plot_simulation_dashboard(
    beliefs_history=agent.history["beliefs"],
    vfe_history=agent.history["vfe"],
    observations=env.history["observations"],
    predictions=[model.A @ b for b in agent.history["beliefs"]],
    efe_history=agent.history.get("efe", []),
    save_path="output/lab08_dashboard.png",
)
```

Applications

- **Robot navigation**: Gridworld planning maps directly to physical navigation tasks.
- **Delayed gratification**: Agents that plan ahead can forgo immediate rewards for larger future rewards.
- **Hierarchical planning**: Multiple levels of temporal depth enable abstract plans (goals) and concrete actions.

Conclusion

Planning extends Active Inference from reactive to deliberative behavior. By evaluating multi-step policies through accumulated EFE and using MMP for deep state inference, agents can handle complex environments with delayed rewards and long horizons. This completes the 8-module Computational Active Inference course — from systems and agents through perception, cognition, action, learning, communication, and planning.